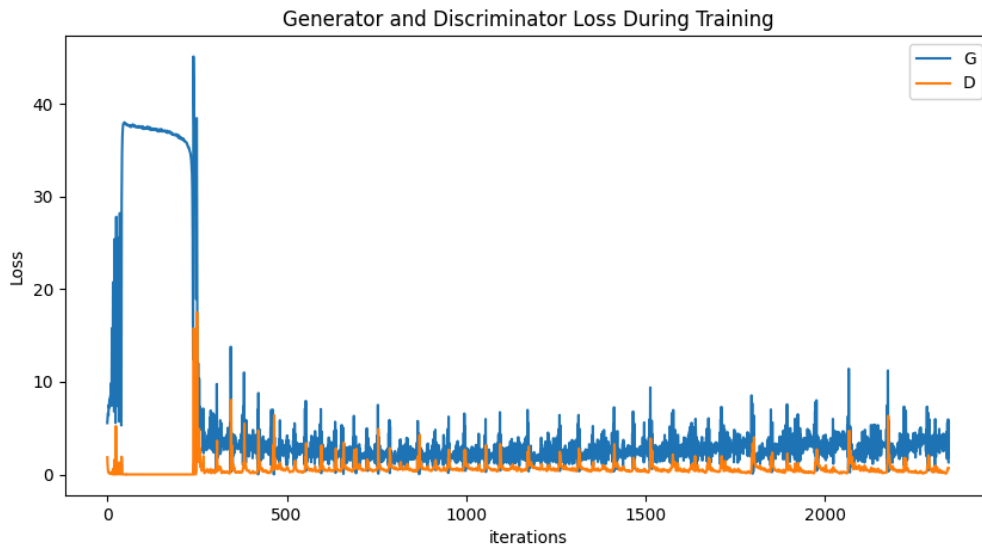


Lab7实验报告

李毅PB22051031

运行结果如下：

Loss：



1. 输入噪声的维度是多少?原始图像的维度是多少? 生成图像的维度是多少?

Z_DIM = 100

噪声向量的维度（输入给生成器）

输入噪声的维度为100

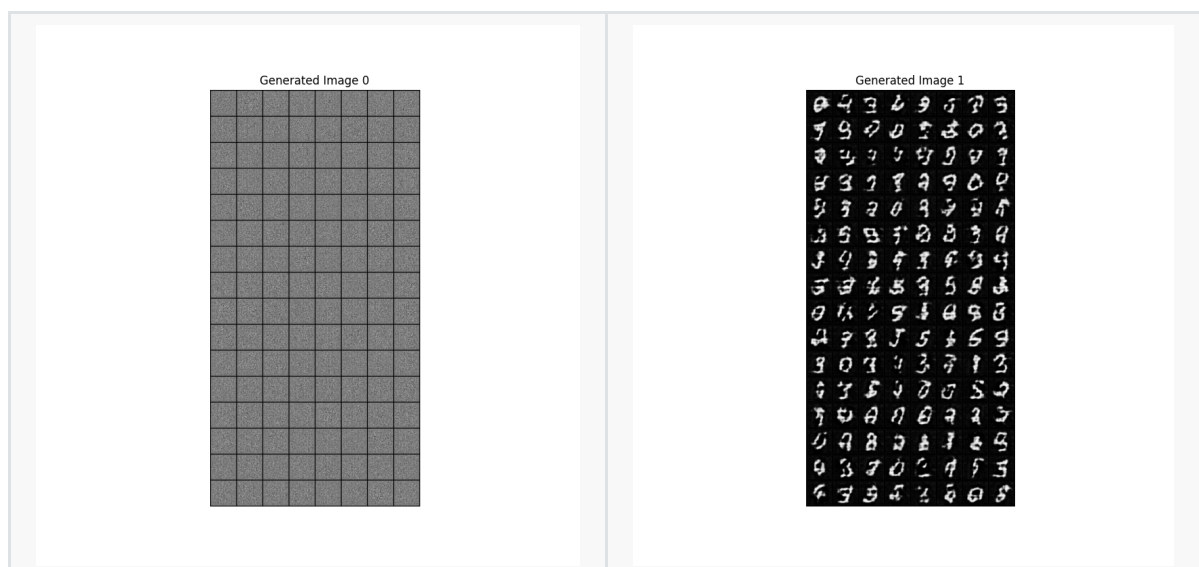
原始下载的 MNIST 是单通道 28*28

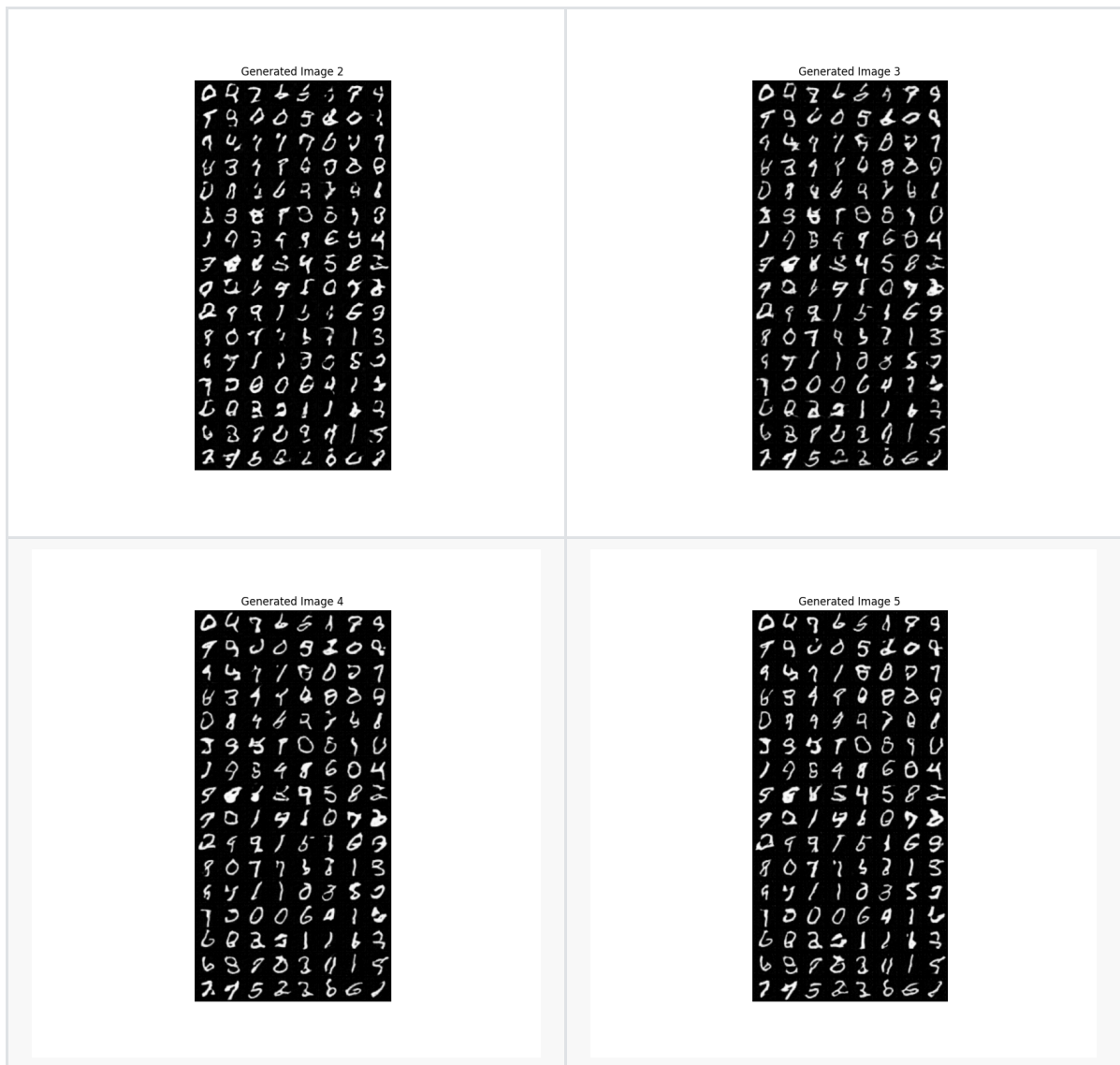
X_DIM = 64

图像的尺寸（宽高均调整为64）

生成图像的维度是64*64

2. 通过查看img_list观察生成效果是不是在不断地变好?





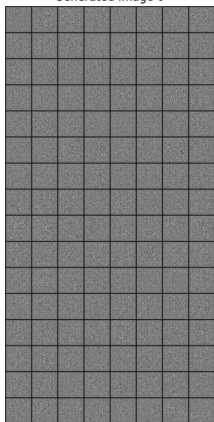
观察发现，img_list生成效果在不断地变好。

3. 如果我们想生成倒立的MNIST数字，怎么办？

```
dataset = dset.MNIST(root=DATA_PATH, download=True,
                    transform=transforms.Compose([
                        transforms.Resize(X_DIM),
                        transforms.ToTensor(),
                        transforms.RandomVerticalFlip(p = 1.0), # 随机水平翻转
                        transforms.RandomHorizontalFlip(p = 1.0), # 随机垂直翻转
                        transforms.Normalize((0.5,), (0.5,))
                    ]))
```

对数据集先水平翻转再垂直翻转（即将数字倒立），即可生成。结果如下：

Generated Image 0



Generated Image 1



Generated Image 2



Generated Image 3



Generated Image 4



Generated Image 5



成功生成了倒立的MNIST数字

4. 如果我们想直接生成32*32的图像，那么代码要如何修改？

将 `X_DIM` 修改为32

```
X_DIM = 32
```

```
# 图像的尺寸（宽高均调整为32）
```

Generator减少一层hidden, Discriminator最后一层卷积核从 4x4 改为 2x2, 使其与当前特征图匹配。

```
# Generator: 根据随机噪声向量生成图像
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            # input layer and前两层hidden保持不变
            nn.ConvTranspose2d(Z_DIM, G_HIDDEN * 8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(G_HIDDEN * 8),
            nn.ReLU(True),
            nn.ConvTranspose2d(G_HIDDEN * 8, G_HIDDEN * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(G_HIDDEN * 4),
            nn.ReLU(True),
            nn.ConvTranspose2d(G_HIDDEN * 4, G_HIDDEN * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(G_HIDDEN * 2),
            nn.ReLU(True),
            # output layer: 直接生成32x32单通道图像
            nn.ConvTranspose2d(G_HIDDEN * 2, IMAGE_CHANNEL, 4, 2, 1, bias=False),
            nn.Tanh()
        )

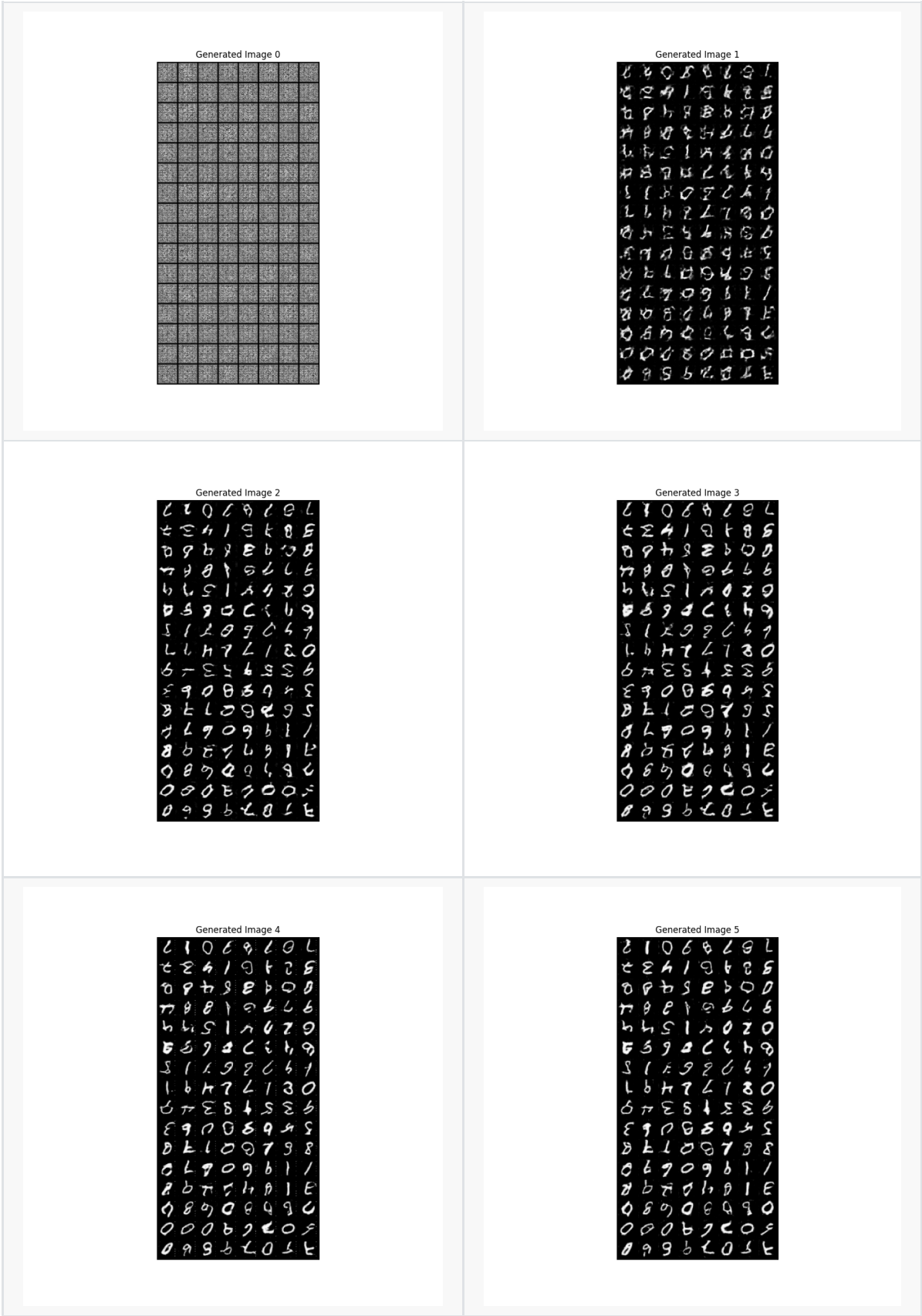
    def forward(self, input):
        return self.main(input)

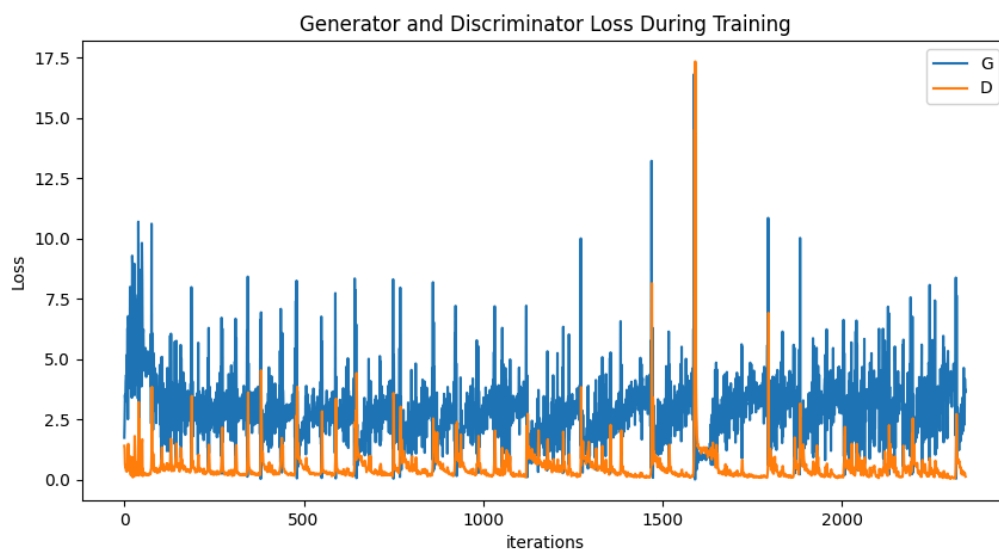
# ----- 定义判别器网络 -----
# Discriminator: 对输入图像（真实或生成的）进行真假判定
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            # 1st layer
            nn.Conv2d(IMAGE_CHANNEL, D_HIDDEN, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # 2nd layer
            nn.Conv2d(D_HIDDEN, D_HIDDEN * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(D_HIDDEN * 2),
            nn.LeakyReLU(0.2, inplace=True),
            # 3rd layer
            nn.Conv2d(D_HIDDEN * 2, D_HIDDEN * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(D_HIDDEN * 4),
            nn.LeakyReLU(0.2, inplace=True),
            # 4th layer
            nn.Conv2d(D_HIDDEN * 4, D_HIDDEN * 8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(D_HIDDEN * 8),
            nn.LeakyReLU(0.2, inplace=True),
            # output layer
            nn.Conv2d(D_HIDDEN * 8, 1, 2, 1, 0, bias=False), # kernel 2 for
32x32 input
            nn.Sigmoid() # 输出为0到1之间, 表示真假概率
        )

    def forward(self, input):
        return self.main(input).view(-1, 1).squeeze(1)
```

结果如下:

img_list:





注：此时仍然启用了任务3中设置，即生成倒立的MNIST数字。